



SMART TEA STALL MANAGEMENT SYSTEM

Project Architecture, Features, Workflow & Implementation Guide

Technology	Java • JDBC • MySQL (Console Application)
Architecture	Model • DAO • Service • Command-Line Interface
Interface	Command-Line Interface (CLI) only
Purpose	Tea-stall ordering and management operations

Prepared as a clear technical/presentation document — current CLI-only build

1. Executive Summary

The Smart Tea Stall Management System is a Java-based database application that manages customer ordering and day-to-day tea-stall operations. It uses MySQL as the persistent data layer, JDBC for database connectivity, and Java classes as the application layer. The user interface is a command-line application; there is currently no graphical (JavaFX) interface in this build.

The implementation is organized into separate packages so that data representation, database access, and business logic remain easy to understand and extend. The database contains tables for menu data, orders, payments, seating, employees, inventory, suppliers, assets, expenses, utility bills, maintenance, damages, attendance, leave, feedback and administration.

Important project status: The project has a fully working command-line application: guided customer ordering (info → table selection → category-wise menu browsing → cart → order → payment → feedback), a complete 20-option admin dashboard covering every operational area, and a MySQL database. There is no JavaFX/FXML/GUI layer in the current build — all interaction happens through the console.

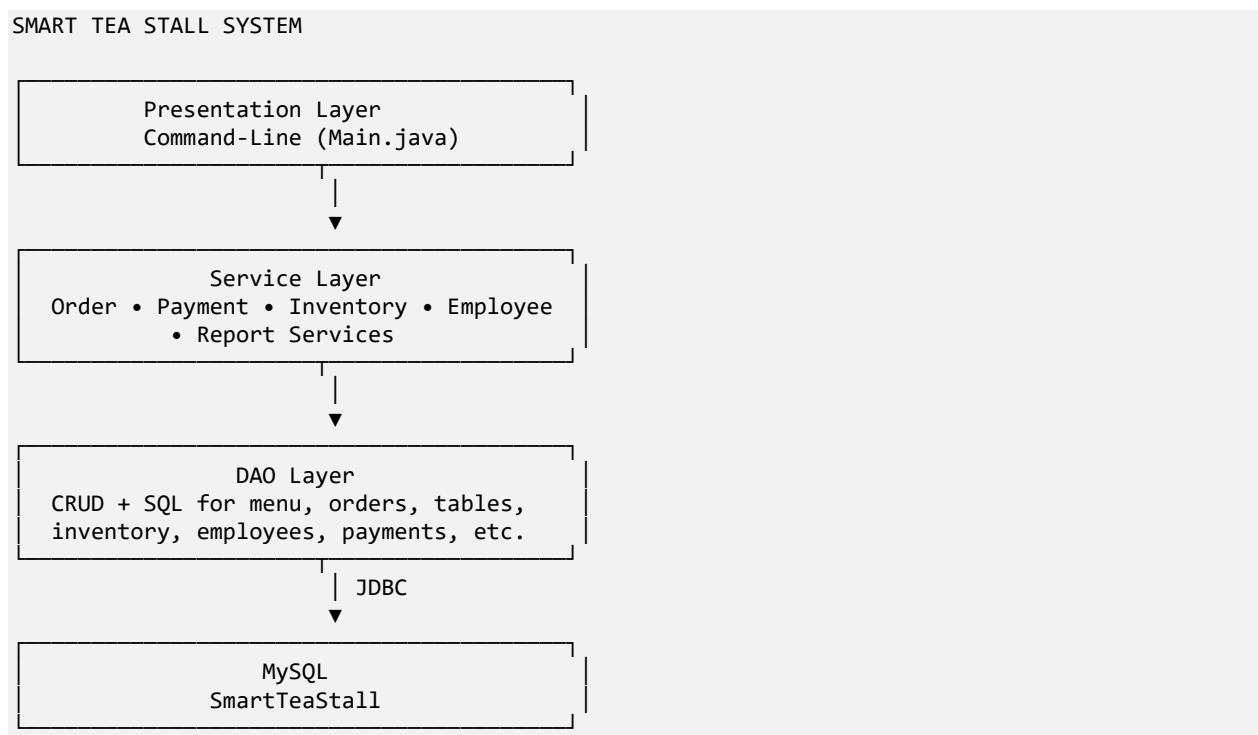
2. Project Objectives

- Provide a single system for managing tea-stall operations and records.
- Store business data permanently in MySQL instead of relying on temporary Java variables.
- Separate database access from business logic using DAO and Service layers.
- Support customer ordering, table selection, cart handling, payment and feedback.
- Support administration of employees, inventory, suppliers, assets and operational expenses.
- Provide reports such as revenue, expenses, profit, order count, employee count and inventory value.
- Deliver all of the above through a single, self-contained command-line interface.
- Keep the design modular (Model/DAO/Service split) so a graphical front end could be added later without rewriting the backend.

3. Technology Stack

Technology	Role	Where Used	Why
Java	Core application language	Models, DAO, Services, Main	Object-oriented structure and JDBC support
MySQL	Database	All persistent business data	Relational storage and SQL querying
JDBC	Database connectivity	DatabaseConnection + DAOs	Connects Java to MySQL
Eclipse	Development IDE	Project structure and execution	Java development and debugging
MySQL Connector/J	JDBC driver	Referenced library	Allows Java to communicate with MySQL

4. Overall Architecture



The key principle is separation of responsibility. A model represents data, a DAO communicates with the database, a service performs reusable business operations/calculations, and Main.java handles all user interaction directly (there is no separate controller/UI layer in this build). This keeps the DAO and Service layers reusable if a graphical interface is added in the future.

5. End-to-End Data Flow

1. User selects an operation in the CLI menu.

2. Main.java collects the required input via the console.
3. The relevant service is called when business logic is needed.
4. The service calls the corresponding DAO.
5. The DAO opens a JDBC connection through DatabaseConnection and executes SQL.
6. MySQL returns rows or confirms an INSERT/UPDATE/DELETE.
7. The DAO converts database rows into model objects.
8. The service returns the result to Main.java.
9. Main.java prints the result to the console.

Example: Display Menu

```
User → Main.java
  → MenuCategoryDAO / MenuItemDAO
  → SELECT ... FROM menu_categories/menu_items
  → MySQL
  → List<MenuCategory> / List<MenuItem>
  → Main.java prints the menu to the console
```

6. Package Structure

Package / Folder	Main Responsibility	Examples
com.smartteastall.database	Database connectivity	DatabaseConnection.java
com.smartteastall.model	Data/model objects	MenuItem, Order, Inventory, Employee
com.smartteastall.dao	SQL and CRUD database access	MenuItemDAO, OrderDAO, InventoryDAO
com.smartteastall.service	Business logic and calculations	OrderService, ReportService
com.smartteastall.main	Application entry point / CLI	Main.java
com.smartteastall.test	Small connectivity/feature tests	ConnectionTest, TableTest, SeatTest

Note: earlier project plans also included com.smartteastall.controller and resources/fxml + resources/css packages for a JavaFX interface. Those are not part of the current build and are intentionally omitted here — see Section 14.

7. File-by-File Explanation

The following list describes the files present in the current project. The purpose is explained in simple presentation language rather than line-by-line code commentary.

7.1 Database Package

File	What it does	Example
DatabaseConnection.java	Creates a JDBC connection to the MySQL database and reports success/failure.	DriverManager.getConnection(...)

7.2 Model Package

Model file	Purpose	Important fields
Addon.java	Represents an optional add-on linked to a menu item.	addonId, itemId, addonName, addonPrice
Admin.java	Represents an administrator account.	adminId, username, password
Asset.java	Represents fixed assets/equipment.	assetId, assetName, category, quantity, price, purchaseDate
Attendance.java	Represents employee attendance.	attendanceId, employeeId, attendanceDate, status
Damage.java	Represents damage records and cost/status.	damageId, itemName, quantity, damageCost, status
Employee.java	Represents staff information and salary components.	employeeId, employeeName, role, phone, salary, bonus, deduction
Expense.java	Represents business expenses.	expenseId, expenseName, category, amount, expenseDate
Feedback.java	Represents customer feedback.	feedbackId, orderId, rating, comments
Inventory.java	Represents stock items and purchase information.	inventoryId, itemName, quantity, unit, minimumStock, purchasePrice
Leave.java	Represents employee leave.	leaveId, employeeId, leaveFrom, leaveTo, reason, status
Maintenance.java	Represents equipment/asset maintenance.	maintenanceId, itemName, problem, repairCost, repairDate, technician
MenuCategory.java	Represents a menu category.	categoryId, categoryName, description
MenuItem.java	Represents an item sold by the stall.	itemId, categoryId, itemName, description, price, available, imagePath
Order.java	Represents a customer order.	orderId, tableId, customerCount, sharingAllowed, orderStatus, totalAmount, orderTime
OrderItem.java	Represents an item inside an order.	orderItemId, orderId, itemId, quantity, unitPrice, subtotal
Payment.java	Represents payment information.	paymentId, orderId, amount, paymentMethod, paymentStatus, paymentTime

Model file	Purpose	Important fields
Seat.java	Represents a seat belonging to a table.	seatId, tableId, seatNumber, status
Supplier.java	Represents a supplier.	supplierId, supplierName, phone, address, suppliedItem
Table.java	Represents a dining table.	tableId, tableName, capacity, status, location
UtilityBill.java	Represents utility bills.	billId, billType, amount, billMonth, billYear, paymentStatus

7.3 DAO Package

DAO file	Purpose	Implemented operations
AddonDAO.java	Create and read add-ons.	addAddon(), getAllAddons()
AdminDAO.java	Admin CRUD and login verification.	addAdmin(), login(), getAdminById(), updateAdmin(), deleteAdmin()
AssetDAO.java	Asset CRUD.	addAsset(), getAllAssets(), getAssetById(), updateAsset(), deleteAsset()
AttendanceDAO.java	Attendance CRUD.	addAttendance(), getAllAttendance(), getAttendanceById(), updateAttendance(), deleteAttendance()
DamageDAO.java	Damage CRUD.	addDamage(), getAllDamages(), getDamageById(), updateDamage(), deleteDamage()
EmployeeDAO.java	Employee creation/listing.	addEmployee(), getAllEmployees()
ExpenseDAO.java	Expense CRUD.	addExpense(), getAllExpenses(), getExpenseById(), updateExpense(), deleteExpense()
FeedbackDAO.java	Feedback creation and listing.	addFeedback(), getAllFeedback()
InventoryDAO.java	Inventory CRUD and stock records.	addInventory(), getAllInventory(), getInventoryById(), updateInventory(), deleteInventory()
LeaveDAO.java	Leave CRUD.	addLeave(), getAllLeaves(), getLeaveById(), updateLeave(), deleteLeave()
MaintenanceDAO.java	Maintenance CRUD.	addMaintenance(), getAllMaintenance(), getMaintenanceById(), updateMaintenance(), deleteMaintenance()
MenuCategoryDAO.java	Menu category creation and listing.	addCategory(), getAllCategories()

DAO file	Purpose	Implemented operations
MenuItemDAO.java	Menu item creation and listing.	addMenuItem(), getAllItems()
OrderDAO.java	Order creation and listing.	addOrder(), getAllOrders()
OrderItemDAO.java	Order-item creation and listing.	addOrderItem(), getAllOrderItems()
PaymentDAO.java	Payment creation and listing.	addPayment(), getAllPayments()
SeatDAO.java	Seat CRUD.	addSeat(), getAllSeats(), getSeatById(), updateSeat(), deleteSeat()
SupplierDAO.java	Supplier CRUD.	addSupplier(), getAllSuppliers(), getSupplierById(), updateSupplier(), deleteSupplier()
TableDAO.java	Table CRUD.	addTable(), getAllTables(), getTableById(), updateTable(), deleteTable()
UtilityBillDAO.java	Utility bill CRUD.	addUtilityBill(), getAllUtilityBills(), getUtilityBillById(), updateUtilityBill(), deleteUtilityBill()

7.4 Service Package

Service	Role	Key functionality
OrderService.java	Coordinates order and order-item operations; calculates total from order-item subtotals.	createOrder(), addOrderItem(), getAllOrders(), getAllOrderItems()
PaymentService.java	Processes payments and derives revenue/payment statistics.	makePayment(), getAllPayments(), getTotalRevenue(), getTodayRevenue(), getSuccessfulPaymentCount()
InventoryService.java	Adds/updates stock and performs stock calculations.	addInventory(), updateInventory(), deleteInventory(), isLowStock(), calculateInventoryValue(), reduceStock(), addStock()
EmployeeService.java	Manages employee records and salary calculations.	addEmployee(), getAllEmployees(), calculateMonthlySalaryExpense()
ReportService.java	Builds management metrics from DAO data.	getTotalRevenue(), getTotalExpenses(), getNetProfit(), getTotalOrders(), getTotalEmployees(), getInventoryValue(), getAverageOrderValue(), printDashboardSummary()

7.5 Main and Test Files

File	Purpose	Example
Main.java	Command-line application entry point. Provides the Customer panel (guided order, table selection, category-wise menu browsing, cart, payment, feedback) and a 20-option Admin dashboard covering menu, tables, seats, employees, inventory, suppliers, assets, expenses, utility bills, maintenance, damage, attendance, leave, feedback and reports.	main() → Customer / Admin / System Information
ConnectionTest.java	Tests whether JDBC can connect to the configured MySQL database.	Connection Test Successful/Failed
TableTest.java	Simple table DAO test used to verify table retrieval/operations.	Table DAO test
SeatTest.java	Simple seat DAO test used to verify seat retrieval/operations.	Seat DAO test
module-info.java	Defines Java module requirements/exports for the modular application.	requires java.sql

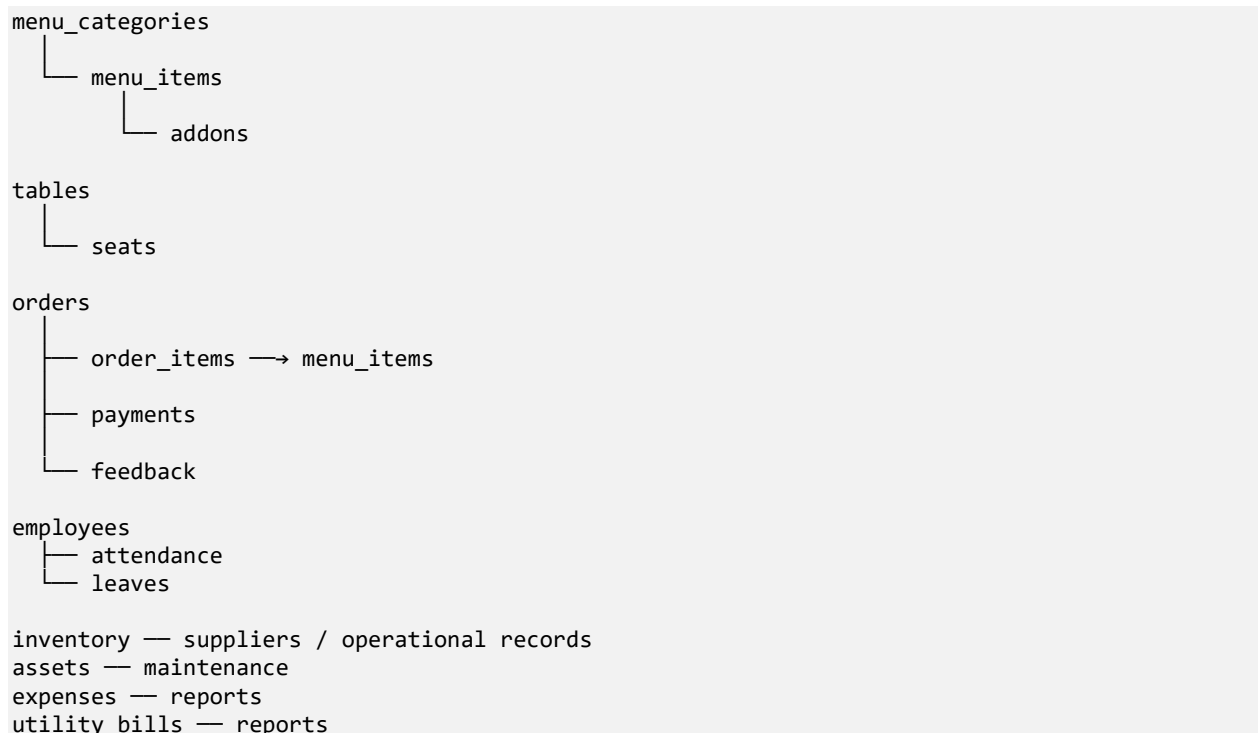
8. Database Design

The project uses the MySQL schema named SmartTeaStall as the persistent store. The database is the source of truth for operational information. Foreign keys are important because child records such as seats depend on their parent tables.

Table	Purpose
admin	Administrator accounts and login data
menu_categories	Menu grouping such as Tea, Coffee, Snacks
menu_items	Products, descriptions, prices and availability
addons	Optional additions linked to menu items
tables	Physical dining tables, capacity, status and location
seats	Individual seats linked to tables
orders	Customer orders and order status
order_items	Items and quantities inside an order
payments	Payment amount, method, status and time
feedback	Customer ratings/comments linked to orders
employees	Staff and salary information
attendance	Employee attendance records
leaves	Employee leave applications/status

Table	Purpose
inventory	Stock quantity, unit, minimum stock and purchase price
suppliers	Supplier contact and supplied-item information
assets	Equipment/fixed assets
expenses	Operational expenses
utility_bills	Electricity, water, internet and other bills
maintenance	Repair and maintenance records
damages	Damage/loss records and cost

9. Database Relationships and Data Dependency



Foreign-key example: A seat cannot be inserted with table_id = 1 unless a parent row with table_id = 1 already exists in the tables table. This is why the database must be populated in dependency order.

Known schema fix: The tables.status column was created as ENUM('AVAILABLE', 'PARTIAL', 'FULL', 'RESERVED'). Main.java originally tried to set status to 'OCCUPIED' when placing an order, which MySQL rejected (Data truncated for column 'status'). This was fixed by using the existing enum value 'FULL' instead of introducing a new one, so no schema change was required.

10. Complete Feature List

Feature	How it is achieved	Main files	Status
JDBC database connection	DriverManager + MySQL Connector/J	DatabaseConnection.java	Implemented
Admin login	DAO login query against admin data	AdminDAO.java	Implemented
Admin CRUD	DAO add/update/delete/get by ID	AdminDAO.java	Implemented
Menu categories	Category model + DAO	MenuCategory.java, MenuCategoryDAO.java	Implemented
Menu items	Item model + DAO	MenuItem.java, MenuItemDAO.java	Implemented
Add-ons	Addon model + DAO	Addon.java, AddonDAO.java	Implemented
Table management	Table model + CRUD DAO	Table.java, TableDAO.java	Implemented
Seat management	Seat model + CRUD DAO	Seat.java, SeatDAO.java	Implemented
Order management	Order + order-item models/DAOs + service	OrderService, OrderDAO, OrderItemDAO	Implemented
Cart calculation	CLI CartLine, category-wise menu browsing + running total	Main.java	Implemented
Payments	Payment model/DAO/service + revenue methods	PaymentDAO, PaymentService	Implemented
Feedback	Feedback model + DAO	FeedbackDAO.java	Implemented
Inventory CRUD	Inventory DAO + service	InventoryDAO, InventoryService	Implemented
Low-stock detection	Quantity compared with minimum stock	InventoryService.java	Implemented
Inventory value	Quantity $\times$ purchase price	InventoryService.java / ReportService.java	Implemented
Stock increase/reduction	Service methods addStock()/reduceStock()	InventoryService.java	Implemented
Employee management	Employee DAO/service	EmployeeDAO, EmployeeService	Implemented
Salary calculations	Salary + bonus – deduction	EmployeeService.java	Implemented
Attendance	Attendance model + CRUD DAO	AttendanceDAO.java	Implemented
Leave management	Leave model + CRUD DAO	LeaveDAO.java	Implemented

Feature	How it is achieved	Main files	Status
Suppliers	Supplier model + CRUD DAO	SupplierDAO.java	Implemented
Assets	Asset model + CRUD DAO	AssetDAO.java	Implemented
Expenses	Expense model + CRUD DAO	ExpenseDAO.java	Implemented
Utility bills	UtilityBill model + CRUD DAO	UtilityBillDAO.java	Implemented
Maintenance	Maintenance model + CRUD DAO	MaintenanceDAO.java	Implemented
Damage tracking	Damage model + CRUD DAO	DamageDAO.java	Implemented
Reports	Revenue, expenses, profit, orders, employees, inventory, average order value	ReportService.java	Implemented
CLI demonstration	Guided customer/admin flow, category-wise menu browsing, live checkout, payment and feedback	Main.java	Fully working end-to-end, tested with live MySQL data

The graphical interface (JavaFX/FXML/CSS) that appeared in the original project plan is not part of this build. See Section 14 for details.

11. Main Use Cases and Workflows

11.1 Customer Ordering Workflow

```

START
↓
Customer Panel
↓
Enter customer name / phone / number of people
↓
Choose sharing preference
↓
Select available table
↓
Browse menu categories
↓
Load menu items from MySQL
↓
Select item + quantity → Add to cart
↓
Review cart / calculate total
↓
Create order + order items
↓
Make payment
↓
Update order/payment status
↓
Optional feedback

```

↓
END

Example:

- Customer: Arun; phone: 9876543210; people: 2; sharing: allowed.
- Available table: T1 with capacity 4.
- Selected items: Milk Tea × 2 at ₹15 each.
- Order total: $2 \times 15 = ₹30$.
- Payment method: UPI; payment status: PAID.
- Feedback: rating 5 with a short comment.

11.2 Admin Workflow

```
Admin Login
↓
Admin Dashboard (console menu)
├── Menu / Categories / Add-ons
├── Tables / Seats
├── Orders / Payments
├── Employees / Attendance / Leave
├── Inventory
├── Suppliers
├── Assets
├── Expenses / Utility Bills
├── Maintenance / Damage
└── Reports
```

11.3 Inventory Workflow

10. Admin selects Inventory from the console menu.
11. InventoryService loads inventory records through InventoryDAO.
12. The service can determine whether an item is low stock.
13. Admin can add stock or reduce used stock from the console.
14. Inventory value is calculated as quantity × purchase price.
15. Updated records remain in MySQL and are available to reports.

Example:

```
Tea Powder
Quantity = 10 kg
Purchase price = ₹450/kg
Inventory value = 10 × 450 = ₹4,500
```

```
If minimum stock = 3 kg and quantity becomes 2 kg:
Low stock = TRUE
```

11.4 Reporting Workflow

```
Payments → total PAID amounts → Total Revenue
Expenses → sum of expenses → Total Expenses
```

```
Net Profit = Total Revenue - Total Expenses
```

```
Orders → count → Total Orders
Employees → count → Total Employees
```

Inventory $\rightarrow \Sigma(\text{quantity} \times \text{purchase price}) \rightarrow \text{Inventory Value}$
Average Order Value = Total Revenue / Total Orders

12. Example Data Lifecycle

The following example shows how a single customer transaction moves through the system once the database is populated and the CLI calls the backend services.

Stage	Example data	Database object	Result
Customer input	Arun, 2 people, sharing allowed	Application session	Input validated
Table	T1, capacity 4, AVAILABLE	tables	Table selected
Menu	Milk Tea ₹15	menu_categories + menu_items	Items displayed from DB
Cart	Milk Tea $\times$ 2	In-memory CartLine list in Main.java	Subtotal ₹30
Order	Order #101, table T1	orders	Order created
Order lines	Item 1 $\times$ 2	order_items	Order details saved
Payment	₹30, UPI, PAID	payments	Payment recorded
Feedback	5/5	feedback	Feedback stored
Reports	Revenue +₹30, orders +1	ReportService	Dashboard reflects transaction

13. SQL as the Source of Truth

A major design goal of this project is that operational details such as menu names, prices, availability, tables and inventory ultimately come from MySQL rather than being permanently hardcoded in Java.

Example menu query:

```
SELECT *  
FROM menu_items  
WHERE available = 1  
ORDER BY category_id, item_name, item_id;
```

- SELECT retrieves current records from the database.
- WHERE filters records, for example only available menu items.
- ORDER BY controls presentation order at the SQL level.
- INSERT creates new records.
- UPDATE changes existing records.
- DELETE removes records where appropriate.
- Foreign keys protect relationships between parent and child tables.

Current implementation note: Main.java's guided customer flow reads menu categories and items live from MySQL through MenuCategoryDAO/MenuItemDAO — menu data is not hardcoded in the CLI. Prices and availability are controlled entirely from the database.

14. Interface: Console-Only (No JavaFX)

The original project plan included a JavaFX/FXML/CSS graphical interface alongside the command-line prototype. In the current build, that graphical layer does not exist — there are no controller classes, no FXML screens, and no CSS styling. All customer and admin interaction happens through the console (System.out / Scanner) in Main.java.

This does not affect the underlying design: because database access lives in the DAO layer and business logic lives in the Service layer, a GUI could still be added later by writing new controller classes that call the same OrderService, PaymentService, InventoryService, EmployeeService and ReportService methods already used by the CLI. No changes to the database, models, DAOs or services would be required to add a graphical front end.

- Current interface: 100% command-line, run from Main.java's main() method.
- Customer interaction: numbered menu options typed into the console.
- Admin interaction: numbered menu options typed into the console, after username/password login.
- Output: formatted plain-text tables printed with System.out.println().

15. Testing and Verification

Test	What it verifies	Expected result
ConnectionTest	JDBC connection to MySQL	Connection Test Successful
TableTest	Table DAO/database operations	Table records can be read/managed
SeatTest	Seat DAO/database operations	Seat records can be read/managed
Guided customer flow (manual)	Info → table → menu → cart → order → payment → feedback	Order and payment are saved and printed back correctly
Admin dashboard (manual)	Each of the 20 admin menu options	CRUD operations succeed and reflect in MySQL
Foreign-key test	Seat references a valid table_id	Invalid parent IDs are rejected by MySQL

16. Current Implementation Status

Area	Current state	Next action
Database schema	Tables created in MySQL; relationships are being populated with data.	Complete seed/master data and verify foreign keys.
Database connectivity	JDBC connection class and connection test are present.	Keep credentials external/secure for production.

Area	Current state	Next action
Models	Domain models are present for major business entities.	Add fields only if later requirements need them.
DAO layer	CRUD/read DAO foundation exists for all entities.	Add missing filtered/search/sorted queries where needed.
Service layer	Order, Payment, Inventory, Employee and Report services exist.	Expand orchestration for complete business transactions.
CLI	Full guided customer flow and 20-option admin dashboard, driven live by MySQL.	Add input validation edge cases and richer error messages.
Graphical interface	Not present in this build. No controller classes or FXML/CSS files exist.	Optional future work — add a GUI layer on top of the existing services.
Security	Admin login DAO exists; password is currently treated as plain text; DB credentials are hardcoded in DatabaseConnection.java.	Hash passwords and move credentials out of source code before any real deployment.
Reports	Core revenue/expense/profit calculations exist.	Connect report output to date/category filters.

17. How to Explain the Project in a Presentation

16. Start with the problem: a tea stall needs one system for orders, seating, inventory, staff and financial records.
17. Show the architecture: Model → DAO → Service → Main (CLI) → MySQL.
18. Explain one customer order from console input to payment.
19. Show that menu prices and operational data belong in MySQL, not hardcoded in Java.
20. Show the DAO as the SQL/database access layer.
21. Show the Service layer for calculations such as order totals, low stock and profit.
22. Walk through the admin dashboard and its 20 management areas.
23. Demonstrate a database update (e.g. change a price in MySQL) and show the CLI reflects it without changing Java code.
24. Finish with future expansion: a JavaFX or web front end reusing the same Service layer, stronger validation/security, and transaction handling.

18. Short Viva / Review Answers

Question	Simple answer
Why use DAO?	To isolate SQL/database access from the rest of the application.
Why use Service classes?	To keep reusable business rules and calculations outside the DAO and CLI.
Why use Model classes?	To represent database/application data as Java objects.

Question	Simple answer
Why MySQL?	It provides persistent relational storage and supports relationships, constraints and SQL queries.
Why JDBC?	It is the standard Java API used here to communicate with MySQL through the Connector/J driver.
Why is there no GUI right now?	The current build focuses on a fully working, testable console application; the layered design (DAO/Service) means a GUI can be added later without changing the backend.
What is a foreign key?	A constraint that links a child record to an existing parent record, such as <code>seats.table_id → tables.table_id</code> .
How is profit calculated?	Total paid revenue minus total expenses.
How is inventory value calculated?	For each item: quantity × purchase price, then sum all items.
How is low stock detected?	Current quantity is compared with the configured minimum stock level.
Why should menu prices be in SQL?	So price changes can be made centrally without recompiling the application.

19. Recommended Completion Plan

25. Finish and verify master data in MySQL: categories, menu items, add-ons, tables and seats.
26. Populate staff, inventory, suppliers, assets and operational records with realistic test data.
27. Connect table selection to TableDAO and enforce availability/capacity rules more strictly.
28. Create a transaction-safe order flow: order + order_items + payment + stock update in one DB transaction.
29. Add search/filter/sort queries where users need them (e.g. orders by date, low-stock alerts).
30. Improve security by hashing admin passwords and removing hardcoded database credentials from source code.
31. Close JDBC connections/resources properly (try-with-resources) in every DAO method.
32. Perform end-to-end testing with realistic database records.
33. (Optional, future) Add a graphical or web front end that calls the existing Service layer.

Appendix A — Example CLI Session

```
-----  
SMART TEA STALL MANAGEMENT SYSTEM  
-----
```

```
Database Connected Successfully.  
-----
```

```
MAIN MENU  
-----
```

- 1. Customer
- 2. Admin
- 3. System Information
- 0. Exit

```
Select option: 1  
-----
```

```
CUSTOMER  
-----
```

- 1. Start New Order
- 2. View Tables
- 3. Browse Menu by Category (Add to Cart)
- 4. View Cart
- 5. Update Cart Quantity
- 6. Remove Cart Item
- 7. Place Order
- 8. Make Payment
- 9. Give Feedback
- 10. View My Order Summary
- 0. Back

```
Select option: 1  
-----
```

```
Customer name: Arun  
Mobile number: 9876543210  
Number of people: 2  
Allow table sharing? (Y/N): y
```

```
Recommended available tables for 2 people:
```

```
-> 1 | T1 | capacity 4
```

```
Enter table ID to select, or 0 to cancel: 1
```

```
Table selected: T1  
-----
```

```
MENU CATEGORIES  
-----
```

- 1 Tea
- 2 Coffee
- 3 Snacks

```
Select a category ID to view items, or 0 to finish browsing: 1  
-----
```

```
TEA  
-----
```

```
1 Milk Tea    ₹15.00
```

```
2 Ginger Tea  ₹20.00
```

```
Enter item ID to add to cart, or 0 to go back: 1
```

```
Quantity: 2
```

```
Added to cart: Milk Tea x 2
```

```
Add another item from Tea? (Y/N): n  
-----
```

```
Select a category ID to view items, or 0 to finish browsing: 0  
-----
```

```
CART  
-----
```

```
Milk Tea  x2  ₹15.00  ₹30.00
```



```
TOTAL: ₹30.00
Proceed to checkout with this cart? (Y/N): y
Place this order? (Y/N): y
Order placed successfully.
Order ID : 1
```

```
PAYMENT
Amount : ₹30.00
1. CASH
2. UPI
3. CARD
Payment method: 2
Payment recorded successfully.
Method: UPI
Amount: ₹30.00
```

```
Would you like to give feedback? (Y/N): y
Rating (1-5): 5
Thank you! Your order is complete.
```

Appendix B — Example SQL Operations

```
-- Display menu in database order
SELECT item_id, item_name, category_id, price
FROM menu_items
WHERE available = 1
ORDER BY category_id, item_name, item_id;

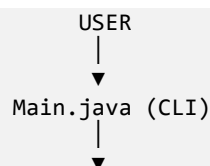
-- Change a menu price
UPDATE menu_items
SET price = 18.00
WHERE item_id = 1;

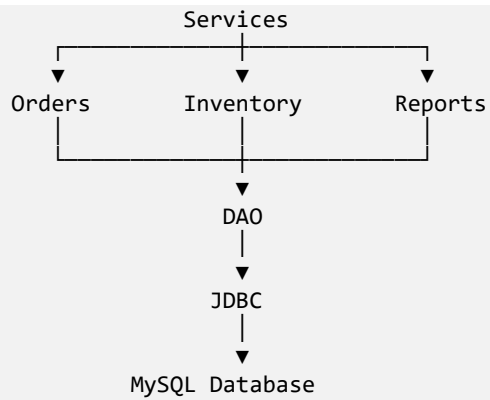
-- Add inventory
INSERT INTO inventory
(item_name, quantity, unit, minimum_stock, purchase_price)
VALUES
('Tea Powder', 10, 'kg', 3, 450.00);

-- Update stock
UPDATE inventory
SET quantity = quantity + 5
WHERE inventory_id = 1;

-- View available tables
SELECT *
FROM tables
WHERE status = 'AVAILABLE'
ORDER BY table_id;
```

Appendix C — Final Concept in One Diagram





Data is stored once in MySQL and served entirely through the console interface.

Final takeaway: The project is a layered management system running entirely on the command line. MySQL stores the business data, DAOs provide database operations, Services apply business rules, and Main.java handles all user interaction through the console. The architecture is ready to support a graphical front end in the future without changing the database foundation.